

Homework 1

Ravi Kini

October 13, 2025

Exercise 1

Part (a)

The following code approximates the exponential function using a Taylor series centered at $x_0 = 0$.

```
1 def factorial(n): # function to compute factorial
2     if n == 0:
3         return 1
4     return n * factorial(n - 1)
5
6 def e(x, n): # compute e^x using first n terms of taylor series centered at x0 = 0
7     return sum(x ** i / factorial(i) for i in range(n))
```

For $n = 10$ terms, the following table shows the approximations and relative errors.

x	Approximation	Relative Error (%)
-30π	-1474668384895.6338	$1.2588818869051297 \cdot 10^{53}$
-0.5	0.6065306594552607	$4.243358099105179 \cdot 10^{-10}$
0.5	1.6487212704182512	$1.7096702227480595 \cdot 10^{-10}$
30π	1785422635167.8518	1.0

For $n = 40$ terms, the following table shows the approximations and relative errors.

x	Approximation	Relative Error
-30π	$-3.432708336229212 \cdot 10^{30}$	$2.930404144938005 \cdot 10^{71}$
-0.5	0.6065306597126334	0.0
0.5	1.6487212707001282	0.0
30π	$8.198519544745587 \cdot 10^{30}$	0.9999999999039616

The approximation performs very well when x is close to $x_0 = 0$, which is where the Taylor series is centered, but performs poorly far away from x_0 . Additionally, using more terms in the approximation provides a better approximation, which is expected as the error is less.

Part (b)

The following code approximates the exponential function using a Taylor series centered at $x_0 = 0$. It additionally exploits the following property of exponentials:

$$e^x = (e^{x/m})^m$$

by mapping values of x where the Taylor series approximation would be inaccurate to the $[-0.01, 0.01]$ range, where it would be accurate.

```

1 def factorial(n): # function to compute factorial
2     if n == 0:
3         return 1
4     return n * factorial(n - 1)
5
6 def e(x, n): # compute e^x using first n terms of taylor series centered at x0 = 0
7     m = 1
8     if abs(x) > 1: # map to [-0.01, 0.01]
9         m = round(x) * 100 # guaranteed to be integer
10        x = x / m # exploit e^x = (e^(x/m))^m
11        return sum(x ** i / factorial(i) for i in range(n)) ** m

```

For $n = 10$ terms, the following table shows the approximations and relative errors.

x	Approximation	Relative Error
-30π	$1.1714112342361038 \cdot 10^{-41}$	$9.354463301429364 \cdot 10^{-13}$
-0.5	0.6065306594552607	$4.243358099105179 \cdot 10^{-10}$
0.5	1.6487212704182512	$1.7096702227480595 \cdot 10^{-10}$
30π	$8.536711709548494e + 40$	$9.352249887737868 \cdot 10^{-13}$

For $n = 40$ terms, the following table shows the approximations and relative errors.

x	Approximation	Relative Error (%)
-30π	$1.1714112342361038e - 41$	$9.354463301429364 \cdot 10^{-13}$
-0.5	0.6065306597126334	0.0
0.5	1.6487212707001282	0.0
30π	$8.536711709548494e + 40$	$9.352249887737868 \cdot 10^{-13}$

This approximation performs far better than the previous one, as it only computes the Taylor series approximation in the interval where it is accurate, then raises it to an integer power, which is within the capabilities of the computer.

Exercise 2

$$\begin{aligned} I(x) &= \int_x^{x+1} \frac{1}{1+s^2} dx \\ &= \arctan(x+1) - \arctan(x) \end{aligned}$$

Part (a)

When x is large, the analytic result in the above form is unsuitable for computation because both terms will be very close to $\frac{\pi}{2}$. While the approximations themselves may be accurate, the approximation of the difference will be inaccurate since they are very close to each other and a small error in the approximation will be large relative to the difference (catastrophic cancellation).

Part (b)

Using the formula for difference of angles:

$$\begin{aligned} I(x) &= \arctan(x+1) - \arctan(x) \\ &= \arctan(\tan(\arctan(x+1) - \arctan(x))) \\ &= \arctan\left(\frac{\tan(\arctan(x+1)) - \tan(\arctan(x))}{1 + \tan(\arctan(x+1))\tan(\arctan(x))}\right) \\ &= \arctan\left(\frac{(x+1) - (x)}{1 + (x+1)(x)}\right) \\ &= \arctan\left(\frac{1}{1+x+x^2}\right) \end{aligned}$$

Since this formula only involves finding the arctangent of a small number, it will be more accurate for large n .

Part (c)

For both of the expressions for $I(x)$, the following table shows the approximations.

x	$\arctan(x+1) - \arctan(x)$	$\arctan\left(\frac{1}{1+x+x^2}\right)$
10^0	0.32175055439664213	0.3217505543966422
10^1	0.00900876529041672	0.009008765290416915
10^2	$9.900009867647164 \cdot 10^{-5}$	$9.900009867666503 \cdot 10^{-5}$
10^3	$9.99000000945216 \cdot 10^{-7}$	$9.990000009986678 \cdot 10^{-7}$
10^4	$9.999000072369313 \cdot 10^{-9}$	$9.99900000001 \cdot 10^{-9}$
10^5	$9.99991200956174 \cdot 10^{-11}$	$9.99990000000001 \cdot 10^{-11}$
10^6	$1.000088900582341 \cdot 10^{-12}$	$9.99999 \cdot 10^{-13}$
10^7	$9.992007221626409 \cdot 10^{-15}$	$9.999999 \cdot 10^{-15}$
10^8	0.0	$9.9999999 \cdot 10^{-17}$
10^9	0.0	$9.99999999 \cdot 10^{-19}$
10^{10}	0.0	$9.999999999 \cdot 10^{-21}$

For large values of x , the original expression fails to accurately compute the difference, getting a result of 0, while the rewritten expression is more accurate.

Exercise 3

Part (a)

The following code implements a simulation of 1 day with a time step of 0.1 s.

```
1 dt = 0.1
2 t = 0
3 Nsteps = 864000
4
5 for j in range(Nsteps):
6     t += dt
```

The resulting absolute error is 0.00000054126, and the relative error is $6.264565820822975 \cdot 10^{-12}$.

Part (b)

The following code implements a simulation of 1 day with a time step of 0.125 s.

```
1 dt = 0.125
2 t = 0
3 Nsteps = 691200
4
5 for j in range(Nsteps):
6     t += dt
```

The resulting absolute error is 0.0, and the relative error is 0.0.

Part (c)

Since 0.125 is a power of 2, it can be represented exactly in binary, while the same cannot be done for 0.1. Consequently, over hundreds of thousands of steps, error accumulates when using a timestep of 0.1 s, while the same does not occur with a timestep of 0.125 s.

Exercise 4

Part (a)

The following code uses the bisection method to find the roots of the polynomial $q_{20}(x) = \prod_{j=1}^{20}(x - j)$ given its coefficients.

```

1 import numpy as np, scipy as sp
2
3 a = [1, -210, 20615, -1256850, 53327946,
4      -1672280820, 40171771630, -756111184500, 11310276995381,
5      -135585182899530, 1307535010540395, -10142299865511450, 63030812099294896,
6      -311333643161390640, 1206647803780373360, -3599979517947607200, 8037811822645051776,
7      -12870931245150988800, 13803759753640704000, -8752948036761600000, 2432902008176640000]
8 q_20 = lambda x: np.polyval(a, x)
9
10 for j in range(1, 21):
11     root = sp.optimize.bisect(q_20, j - 0.1, j + 0.1)

```

The following table shows the approximation of the root and relative error for each of the roots.

j	Approximated Root	Relative Error
1	1.0	0.0
2	2.0000000000014553	$7.276401703387981 \cdot 10^{-13}$
3	2.999999999838474	$5.3841967924067014 \cdot 10^{-11}$
4	4.0000000025713245	$6.428311212533998 \cdot 10^{-10}$
5	4.999999888088495	$2.2382301491354146 \cdot 10^{-8}$
6	6.000001526066625	$2.543443728203015 \cdot 10^{-7}$
7	6.999993030204495	$9.956860635590445 \cdot 10^{-7}$
8	8.000021335629572	$2.6669465839276272 \cdot 10^{-6}$
9	8.99990817084472	$1.0203343582793632 \cdot 10^{-5}$
10	10.001562500001455	0.000156225589897075
11	10.99940134508797	$5.442613586400552 \cdot 10^{-5}$
12	12.00267489040707	0.00022285785722710497
13	12.999285089304609	$5.499615482543956 \cdot 10^{-5}$
14	14.001470297832563	0.00010501024544479189
15	14.985937499626015	0.0009383797559769532
16	16.0011575559809	$7.234201506046329 \cdot 10^{-5}$
17	16.998583587432222	$8.332532887182666 \cdot 10^{-5}$
18	18.00006774915527	$3.7638277930329434 \cdot 10^{-6}$
19	18.999844360024152	$8.191644778693272 \cdot 10^{-6}$
20	20.000004245391754	$2.1226954262657511 \cdot 10^{-7}$

The roots that can be found to ten digits of accuracy are $j = 1, 2, 3$. The accuracy decreases as the value of the root increases, on average, with $j = 14, 15$ being the least accurate.

Part (b)

$$p(x) = \sum_{k=0}^n a_k x^k$$

The polynomial can be treated as a multivariable function $p(\mathbf{a}, x) = \sum_{k=0}^n a_k x^k$. For the j th root r_j :

$$p(\mathbf{a}, r_j) = 0$$

Holding all variables except r_j and a_i constant and taking the total derivative of both sides:

$$\begin{aligned} \frac{\partial p}{\partial r_j} \cdot \frac{\partial r_j}{\partial a_i} + \frac{\partial p}{\partial a_i} \Big|_{(\mathbf{a}, r_j)} &= 0 \\ p'(\mathbf{a}, r_j) \cdot \frac{\partial r_j}{\partial a_i} + r_j^i &= 0 \\ \frac{\partial r_j}{\partial a_i} &= -\frac{r_j^i}{p'(\mathbf{a}, r_j)} \end{aligned}$$

For small perturbations, $\Delta r_j \approx \frac{\partial r_j}{\partial a_i} \cdot \Delta a_i$. The condition number of r_j with respect to a_i is then:

$$\begin{aligned} \kappa &= \frac{|\Delta r_j / r_j|}{|\Delta a_i / a_i|} \\ &= \left| \frac{\Delta r_j}{\Delta a_i} \cdot \frac{a_i}{r_j} \right| \\ &= \left| \frac{\partial r_j}{\partial a_i} \cdot \frac{a_i}{r_j} \right| \\ &= \left| -\frac{r_j^i}{p'(\mathbf{a}, r_j)} \cdot \frac{a_i}{r_j} \right| \\ &= \left| \frac{a_i r_j^{i-1}}{p'(r_j)} \right| \end{aligned}$$

Defining the condition number κ of a root with respect to changes in the polynomial coefficients as:

Part (c)

$$\kappa = \frac{\max_i |a_i r_j^{i-1}|}{|p'(r_j)|}$$

The following table shows the condition number and relative error for each of the roots.

j	Condition Number	Relative Error
1	113.47567	0.0
2	10043.53971	$7.276401703387981 \cdot 10^{-13}$
3	412181.30464	$5.3841967924067014 \cdot 10^{-11}$
4	9842595.54362	$6.428311212533998 \cdot 10^{-10}$
5	156903300.87007	$2.2382301491354146 \cdot 10^{-8}$
6	1686638685.40285	$2.543443728203015 \cdot 10^{-7}$
7	11768547746.44400	$9.956860635590445 \cdot 10^{-7}$
8	72693521636.77195	$2.6669465839276272 \cdot 10^{-6}$
9	314745601635.24306	$1.0203343582793632 \cdot 10^{-5}$
10	992949726686.49198	0.000156225589897075
11	2450567740788.29977	$5.442613586400552 \cdot 10^{-5}$
12	5221444982030.88124	0.00022285785722710497
13	8396196925546.61736	$5.499615482543956 \cdot 10^{-5}$
14	10215716233334.35598	0.00010501024544479189
15	9351633815150.13707	0.0009383797559769532
16	6339896931582.69492	$7.234201506046329 \cdot 10^{-5}$
17	3169466157551.11359	$8.332532887182666 \cdot 10^{-5}$
18	1175885515157.26578	$3.7638277930329434 \cdot 10^{-6}$
19	263861955839.05950	$8.191644778693272 \cdot 10^{-6}$
20	27053054507.49307	$2.1226954262657511 \cdot 10^{-7}$

The entries with the highest condition numbers are also those with the highest relative errors. The condition number does give information about the results from part (a).

Exercise 5

Suppose that:

$$\frac{\|\mathbf{r}_k\|}{\|\mathbf{b}\|} \leq \epsilon_{\text{tol}}$$

Note that $\mathbf{r}_k$ can be rewritten as:

$$\begin{aligned}\mathbf{r}_k &= \mathbf{b} - A\mathbf{x}_k \\ &= A\mathbf{x} - A\mathbf{x}_k \\ &= A(\mathbf{x} - \mathbf{x}_k)\end{aligned}$$

ince A is nonsingular, there exists some A^{-1} such that:

$$\begin{aligned}A^{-1}\mathbf{r}_k &= A^{-1}A(\mathbf{x} - \mathbf{x}_k) \\ &= \mathbf{x} - \mathbf{x}_k\end{aligned}$$

Then:

$$\begin{aligned}\|A^{-1}\mathbf{r}_k\| &= \|\mathbf{x} - \mathbf{x}_k\| \\ \|A^{-1}\| \cdot \|\mathbf{r}_k\| &\geq \|\mathbf{x} - \mathbf{x}_k\|\end{aligned}$$

Similarly, $\|\mathbf{b}\| \leq \|A\| \cdot \|\mathbf{x}\|$, and so $\frac{\|A\|}{\|\mathbf{b}\|} \geq \frac{1}{\|\mathbf{x}\|}$. Then:

$$\begin{aligned}\frac{\|\mathbf{x} - \mathbf{x}_k\|}{\|\mathbf{x}\|} &\leq \frac{\|A^{-1}\| \cdot \|\mathbf{r}_k\|}{\|\mathbf{x}\|} \\ &\leq \|A^{-1}\| \cdot \|\mathbf{r}_k\| \cdot \frac{\|A\|}{\|\mathbf{b}\|}\end{aligned}$$

Since $\kappa(A) = \|A\| \cdot \|A^{-1}\|$:

$$\begin{aligned}\frac{\|\mathbf{x} - \mathbf{x}_k\|}{\|\mathbf{x}\|} &\leq \kappa(A) \frac{\|\mathbf{r}_k\|}{\|\mathbf{b}\|} \\ &\leq \kappa(A) \epsilon_{\text{tol}}\end{aligned}$$